ASSIGNMENT 4

Relational and NoSQL Databases

Event Ticketing and Venue Seating System

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	23 October 2026

Basic difference

A relational database stores records in tables linked by keys. MySQL is one example. A NoSQL database may store documents, key-value pairs, wide-column data or graphs. MongoDB and DynamoDB are examples, but they do not work in exactly the same way.

Comparison

Point	Relational database	NoSQL database
Structure	Tables have defined columns and data types. Relationships are stored using keys.	The structure depends on the model. A document can contain nested fields; a key-value entry is accessed through its key.
Scalability	Can use larger servers, replicas and sharding. Scaling across servers needs planning, especially for joins.	Many products are built to divide data across servers. A good partition key is still important.
Consistency	Transactions and constraints help keep related changes correct. Replica reads can lag behind recent writes.	Consistency depends on the product and settings. Some support strong reads and multi-record transactions.
Performance	Suitable for linked records, transactions and SQL reports. Indexes help frequent searches.	Can work well for fast key-based access or a known document structure. It is not automatically faster for every task.
Typical uses	Bookings, inventory, orders and accounts.	Flexible catalogues, session storage, large event logs and other access-pattern-specific workloads.

What matters in ticket booking

The important question is whether two people can receive the same seat. The final allocation needs a reliable uniqueness rule and a transaction. A slightly old event description is less serious than an incorrect seat allocation. This means different parts of a large ticketing website may have different storage requirements.

Why companies choose different databases

A company considers the relationships in its data, the queries it needs, expected traffic and the cost of running the system. It also considers how much delay in updated data is acceptable. A system with many related transactions may benefit from relational tables, while a service mainly reading items by a known key may suit a key-value database.

There is also a practical side. The team needs to understand backups, recovery and monitoring. Adding another database should solve a real problem, because it adds more work to maintain.

Real-world examples

Shopify: Shopify has described using MySQL databases divided into shards by shop. This is an example of a relational system being spread across servers as it grows. Relational databases are not limited to one large machine.

Amazon: AWS described DynamoDB handling very high request volumes for Amazon services during Prime Day 2024. This is an example of a NoSQL service supporting a large, changing workload. It does not mean that every Amazon service uses the same database.

In a ticketing application: MySQL can store bookings and payments. A separate cache could hold popular event pages, while flexible event descriptions could be stored as documents if the application became much larger. The booking process would still need a reliable check before assigning a seat.

Choice for this project

MySQL is suitable because Users, Events, Seats, Bookings and Payments are closely related. Primary Key and Foreign Key constraints help connect the records correctly. SQL joins can produce ticket lists and payment reports, and the unique event-seat pair prevents duplicate allocations.

For this college project, one MySQL database is enough. It keeps the implementation easier to understand and supports all the required reports.